



Introduction to Databases

PostgreSql Labs

Author Dr. NAIT-HAMOUD

Department of Foundation training - Second year
National School of Cybersecurity
Algiers, Algeria

Nov 30, 2025

Contents

1	Introduction	3
2	Data Definition Language (DDL)	3
2.1	Create the database and its tables using psql tool	4
2.2	Dropping tables	5
2.3	Tuples insertion using psql tool	5
2.4	Required work –Schema updating	5
2.5	Views	5
2.5.1	Regular vs. materialized views	5
2.5.2	Required work	6
3	Data Manipulation Language (DML)	7
3.1	Querying the database (Required work)	7
4	SQL user-defined Functions and Transactions	9
4.1	SQL user-defined functions	9
4.1.1	SQL functions example	9
4.1.2	Required work	10
4.2	Transactions	10
4.2.1	Required work	10
5	Triggers	11
5.1	Introduction	11
5.2	Example of a statement trigger	12
5.3	Required work	12
6	Project description	14
6.1	Part I	14
6.2	Part II	14
6.2.1	ERD design and Normalization	14
6.2.2	Back to the ultimate database schema before section 6.2.1	14
6.2.3	Python application	15
6.3	Delivrables	15
7	ANNEXES	16
7.1	Some PostgreSQL data types	16
7.2	ANNEX I - Tables to create using psql tool	17
7.3	Remarks	18
7.4	ANNEX II - Tuples insertion	18
7.4.1	Try the following commands.	18
7.4.2	Remarks	20

Document organization

- Highlighted in red : The required work to provide in each session.
- Highlighted in blue: Important remarks about the psql commands.
- In black: work plan and useful information to master PostgreSQL.

Lab 1- Data Defintion Language

Lab goals

- Master DDL language
- Create a schema
- Update the schema
- Insert tuples

1 Introduction

PostgreSQL is a powerful, open-source, object-relational database management system known for its reliability, feature-richness, and flexibility. It supports both SQL and non-relational querying, making it a versatile choice for a wide range of applications, from web and mobile apps to complex data analytics . Developed at the University of California, Berkeley, it has over 30 years of active community development. Key features and characteristics.

- **Object-relational:** It extends the relational model to include object-oriented features, allowing the handling of complex data structures.
- **ACID compliant:** It provides strong data integrity for transactional workloads.
- **Concurrent users:** It is designed to handle many concurrent users without slowdowns through multiversion concurrency control (MVCC).
- **Advanced SQL support:** It supports modern SQL features like foreign keys, and triggers.
- **Data type flexibility:** Beyond traditional relational data, it supports data types like JSON, arrays, and key-value pairs.
- **Open-source and free:** It is available under the permissive PostgreSQL License, with no licensing costs.

PostgreSQL's popularity is driven by the following key strengths:

- **Scalability:** Can handle large and complex datasets and is designed for high performance.
- **Flexibility:** Its object-relational nature and support for various data types make it highly versatile.
- **Strong community:** It is supported by a dedicated global development group and a large community of users and developers.

2 Data Definition Language (DDL)

1. Database creation
2. Tables creation
3. Dropping tables
4. Tuples insertion
5. Schema updating

Method 1: :Using pgAdmin (Graphical User Interface) For those who prefer a visual approach, pgAdmin provides a user-friendly interface:

1. Log in to your PostgreSQL server via the pgAdmin interface.
2. In the Object Explorer, right-click on the Databases menu.
3. Select Create > Database....
4. In the "Create - Database" dialog box, enter the desired database name, owner, and configure any other parameters like encoding or connection limits.
5. Click Save (or OK) to create the new database.

Method 2: Steps:

1. Execute the CREATE DATABASE SQL command inside the psql tool (see figure 1) prompt:

```
CREATE DATABASE my_new_database;
```

2. Verify the creation by listing all databases using the \l command:

```
\l
```

3. Connect to the new database using the \c command to start running further SQL commands (e.g., creating tables) within it:

```
\c my_new_database
```

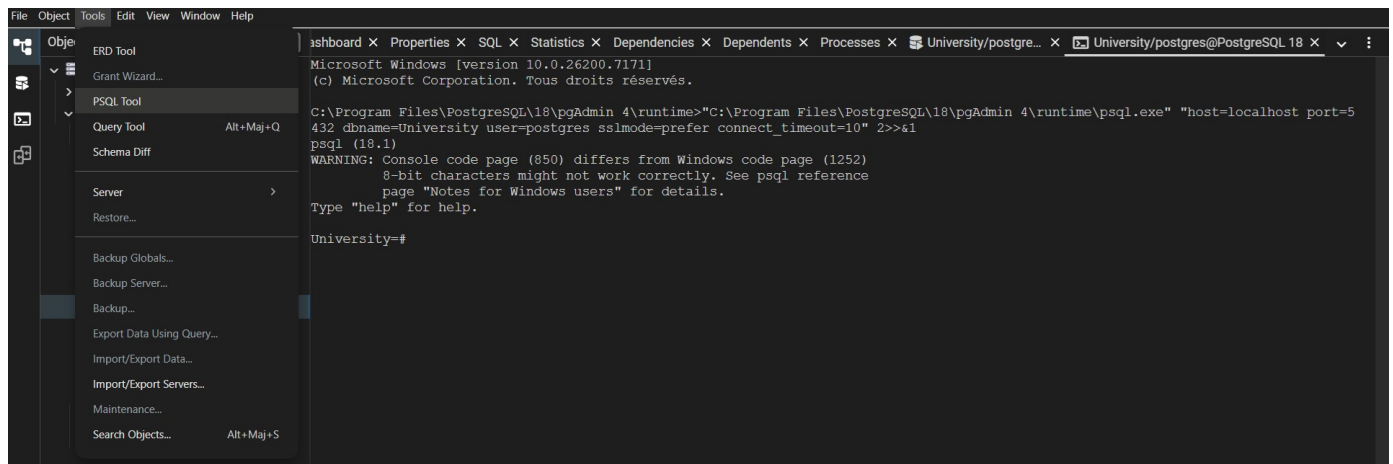


Figure 1: Launching the PSQL Tool

In the following we will use psql tool (Method 2) for the database and tables creation, tables dropping and tuples insertion.

2.1 Create the database and its tables using psql tool

1. Create University databases.
2. Connect to the databases.
3. Create the tables using psql tool (**See Annex I for the commands**).
 - use the psql command `\dt` to list the relational tables in the current database schema.
4. Reverse engineer the schema and provide the ERD to get a global view of the database.

2.2 Dropping tables

- DROP TABLE Student;
- DROP TABLE Reservation;
- DROP TABLE Room;
- DROP TABLE Course;
- DROP TABLE Instructor;
- DROP TABLE Department;

2.3 Tuples insertion using psql tool

Insert tuples into the different tables using psql tool (see **Annex II for the commands**)

- use the \t to turn off displaying columns and allows to display only the raw data (tuples), which is very useful for scripting, copy/pasting data, or exporting results.

2.4 Required work –Schema updating

1. Add a relation that manages students' enrollments in courses (should include the date of enrollment)
2. Add a relation that manages students' marks in the different courses (A student may have more than one mark in each course)

2.5 Views

A view is a virtual table based on the result set of a stored SQL query. It does not store data itself but dynamically shows the query's results when accessed, making it appear like a regular table. Views are useful since they allow the following:

Query simplification: You can save complex or frequently used queries into a view to simplify your SQL code.

Security and access control: Views can restrict access to sensitive columns or rows, allowing users to see only necessary data without exposing the entire underlying table.

Data abstraction: Views can hide changes in the underlying table structure from users and applications.

2.5.1 Regular vs. materialized views

- Regular views: Are dynamic. The underlying query runs every time you query the view, ensuring you always see the latest data. This can be slower for very complex queries.
- Materialized views: Are physical. The query results are stored and can be refreshed periodically. Querying a materialized view is much faster because the data is pre-computed.
- SQL code for creating a view:
The syntax is: `CREATE VIEW view_name AS SELECT query`
- **Example 1:**
`CREATE VIEW GreatSalaries AS SELECT * FROM employees WHERE salary ≥ 50000;`

- **Example 2:**
CREATE MATERIALIZED VIEW avg_Physics_salary AS
SELECT dep_name, AVG(salary) AS average_salary
FROM instructor
WHERE dep_name = 'Physics'
GROUP BY dep_name;

2.5.2 Required work

- Create a regular view to visualize the number of reservations per teacher.
- Create a materialized view to visualize the number of reservations per teacher.

Lab 2- Data Manipulation Language

Lab goals

- Master DML language
- Querying the database

3 Data Manipulation Language (DML)

3.1 Querying the database (Required work)

Write and execute the following SQL queries on the sample data example database.

1. List of the last names and first names of the students stored in the database.
2. List of the last names and first names of the students who live in a chosen city (by you) from the list of cities in the database.
3. List of the last names and first names of the students whose last name starts with 'A'.
4. List of the last names and first names of the teachers whose second-to-last letter of the last name is 'E'.
5. List of the last names and first names of the teachers sorted by department name, then by last name, and then by first name.
6. How many teachers have the grade 'Supleant'?
7. What are the last names and first names of the students who do not have a Fax number (NULL value)?
8. What are the titles of the courses whose description includes the mention 'Licence'?
9. If we assume that one teaching hour costs 3000 DA, what is the cost in DA of each course (course hours concern reserved hours – see the Reservation relationship)?
10. From the previous query, indicate the titles of the courses whose cost is between 3000 and 5000 DA.
11. What are the average capacity and the maximum capacity of the rooms?
12. What are the rooms whose capacity is less than the average capacity?
13. What are the last names and first names of the teachers belonging to the departments named 'SADS' or 'CCS'? (Use IN then another solution)
14. What are the last names and first names of the teachers belonging neither to the 'SADS' department nor to the 'CCS' department?
15. Sort the students by city.
16. How many courses are associated with each department?
17. What are the names of the departments where the number of associated courses is greater than or equal to 3?
18. What are the last names and first names of the teachers for whom there is at least two reservations? (Use EXISTS then another solution using the view created previously).
19. Which teachers have the most reservations (Use the View defined in question 18 and the keyword ALL)?
20. What are the last names and first names of the teachers who do not have any reservations?
21. Which rooms were reserved on all dates (stored in the database)?

22. On which dates are all rooms reserved?
23. Provide 5 examples including **update** clause.
24. Provide 5 distinct examples of aggregation including aggregation with grouping.
25. Provide 5 distinct examples of set operations.
26. Provide 5 examples of querying inside **from** clause.

Lab 3- User-defined SQL functions and Transactions

Lab goals

- Understand the use of user-defined SQL functions and,
- Transactions.

4 SQL user-defined Functions and Transactions

4.1 SQL user-defined functions

Definition. A PostgreSQL function is a stored routine that executes a set of SQL statements and returns a value (which can be a single value, a set of rows, or void). They encapsulate logic, allow code reusability, and enhance performance by reducing network overhead.

PostgreSQL includes two types of functions: built-in (internal) functions and user-defined functions.

- **Built-in Functions:** These are internal, high-performance C-language functions that are part of the PostgreSQL core distribution. They cover a wide range of operations, including mathematical calculations, string manipulation, and date/time processing.
- **User-Defined Functions:** Created by users to perform specific operations, they can be written in several languages:
 - **SQL functions:** Written in standard SQL for simple operations.
 - **Procedural Language functions (PL):** Written in languages such as PostgreSQL's native procedural language (PL/pgSQL), PL/Python, etc. These function allow for enhanced control structures and complex logic.
 - **C-language functions:** C code compiled functions, that are loaded dynamically into the server to enhance performances.

4.1.1 SQL functions example

- Create a function:
CREATE FUNCTION add(a integer, b integer) **RETURNS** integer **AS** \$\$
SELECT a + b;
\$\$ LANGUAGE SQL;
- Call the created function within sql:
SELECT add(6, 9);
- Referencing Parameters with Positional Notation

When a function is defined in PostgreSQL, the parameters are assigned positional identifiers automatically. These identifiers allow for referencing the input values within the function's.

- **\$1:** Stands for the first parameter passed to the function.
- **\$2:** Refers to the second parameter.
- **\$n:** Stands for to the n-th parameter.

– **Example:**
CREATE FUNCTION calculate(t real) **RETURNS** real **AS** \$\$
 – \$1 refers to the 't' parameter
SELECT \$1 * 0.06;
 \$\$ **LANGUAGE** sql;

4.1.2 Required work

1. Write, and show a use case within an SQL query, an SQL function that returns all rooms with a capacity greater than a given value.
2. Write, and show a use case within an SQL query, an SQL function that returns the ID of a department given its name.
3. Write, and show a use case within an SQL query, an SQL function that checks if a room reservation is possible while providing the list of existing reservations (IDs) in conflict with the given reservation's input parameters.

The signature of the function should be as follows:

FUNCTION CheckReservation(param1, param2, param3, param4, param5) **RETURNS** integer

- param1: of type text and represents the Building
- param2: of type text and represents RoomNO
- param3: of type date and represents the date of the reservation to check
- param4: of type time and represents the start_time
- param5: if type time and represents the end_time of the reservation.

4.2 Transactions

Definition:(All-or-nothing). A transaction is a discrete unit of work including a sequence of read or update operations that conclude with a definitive action.

- **COMMIT:** Validates all operations permanently.
- **ROLLBACK:** Cancels all operations, returning to the initial state.

```
BEGIN;
INSERT INTO product_orders (product_id, quantity) VALUES (1, 2);
SAVEPOINT first_insert;
- - Attempt operation that might fail or be incorrect
INSERT INTO product_orders (product_id, quantity) VALUES (2, 5);
- - Roll back to the savepoint.
ROLLBACK TO first_insert;
INSERT INTO product_orders (product_id, quantity) VALUES (3, 1);
- - Commit the whole transaction.
COMMIT;
```

4.2.1 Required work

Considering the university database provided schema,

1. Propose and write two simple transactions without save points.
2. Propose and write two transactions with save points.

Lab 4- Triggers

5 Triggers

5.1 Introduction

- A trigger is a mechanism that enables the automatic execution of a function whenever a specific event occurs on a table or a view.
- This automation is most commonly tied to Data Manipulation Language (DML) operations such as when data is inserted, updated, deleted.
- They offer further flexibility by being able to respond to Data Definition Language (DDL) events as well.

In PostgreSQL, triggers are implemented using functions. A trigger in PostgreSQL isn't the code itself; it acts as an event listener. Its role is to link a specific database event (e.g., an INSERT, UPDATE, or DELETE) to a dedicated function that holds the actual logic to be executed. The process involves two distinct steps:

- **Defining the Trigger Function:** You first create a function using a procedural language like PL/pgSQL. This function contains the necessary logic and must be specifically designed to return the data type TRIGGER, accepting no arguments.
- **Linking the Trigger:** You use the CREATE TRIGGER SQL statement to bind the defined function to a specific table, specifying the timing (BEFORE or AFTER), the event, and the scope (row-level or statement-level).

PostgreSQL triggers are fundamentally categorized by:

- Timing (when they execute relative to the event),
 - Scope (how frequently they execute during a single SQL command).
1. **Timing (When the Trigger Fires):** Timing determines the execution order relative to the INSERT, UPDATE, DELETE, or TRUNCATE statement.

<i>Timing Type</i>	<i>Description</i>	<i>Use Cases</i>
BEFORE	Executes prior to the database operation.	Data validation, calculating derived values, or skipping the operation (by returning NULL).
AFTER	Executes after the database operation is complete and constraints are validated.	Auditing changes, logging historical data, or cascading actions to other tables.
INSTEAD OF	Executes in place of the operation (only used for views).	Allows complex, non-updatable views to accept DML statements by providing custom underlying logic.

2. **Scope/Granularity (Frequency of Firing):** Scope defines how many times the trigger function runs during a single SQL statement.

<i>Scope type</i>	<i>Description</i>	<i>Key feature</i>
FOR EACH ROW (Row-Level)	Fires once for every single row affected by the DML statement.	Accesses OLD (original data) and NEW (new data)
FOR EACH STATEMENT (Statement-Level)	Fires once per SQL command, regardless of the number of affected rows (even zero).	More efficient for bulk operations; cannot access OLD or NEW variables directly.

5.2 Example of a statement trigger

Try the following code.

CREATE TABLE Products(product_id SERIAL PRIMARY KEY , description TEXT, price NUMERIC(10, 2) NOT NULL CHECK (price ≥ 0), category TEXT);
CREATE TABLE last_update_log (table_name VARCHAR(100), last_modified TIMESTAMP);
– Insert an initial row for the Products table INSERT INTO last_update_log (table_name, last_modified) VALUES ('Products', '2015-01-01 00:00:00');
CREATE OR REPLACE FUNCTION update_products_log() RETURNS TRIGGER AS \$\$ BEGIN – This operation happens once per statement. UPDATE last_update_log SET last_modified = CURRENT_TIMESTAMP WHERE table_name = 'Products'; – Statement triggers must return NULL RETURN NULL ; END ; \$\$ LANGUAGE plpgsql ;
CREATE TRIGGER log_products_update_statement_trigger AFTER INSERT OR UPDATE OR DELETE ON Products FOR EACH STATEMENT EXECUTE FUNCTION update_products_log();

- Initial State

SELECT * FROM last_update_log **WHERE** table_name = 'Products';

table_name	last_modified
Products	2015-01-01 00:00:00

- A single DML statement that affects MULTIPLE rows

UPDATE Products **SET** price = price * 1.05 **WHERE** category = 'Electronics';

- Check the log: The function ran only ONCE after the UPDATE statement completed

SELECT * FROM last_update_log **WHERE** table_name = 'Products';

table_name	last_modified
Products	2025-12-15 21:53:00

5.3 Required work

- implement a PostgreSQL statement-level trigger that audits data manipulation operations (INSERT, UPDATE, and DELETE) on the Student table following the steps below.

1. create and use the following table:

```
CREATE TABLE Student_Audit_Log (  
  LogID SERIAL PRIMARY KEY,  
  OperationType VARCHAR(50) NOT NULL,  
  OperationTime TIMESTAMP NOT NULL,  
  Description TEXT  
);
```

2. Requirements

- Create a Trigger Function: Define a PL/pgSQL function named `audit_student_changes_statement()` that records an entry into the `Student_Audit_Log` table after any `INSERT`, `UPDATE`, or `DELETE` statement runs on the `Students` table.
- Log Details: The function must insert a single row into the audit table containing: `OperationType`: The type of SQL operation performed (e.g., 'INSERT', 'UPDATE', 'DELETE'). `OperationTime`: The current timestamp (`CURRENT_TIMESTAMP`).
- Description: A generic message indicating the action, such as 'A statement-level DML operation occurred on Students table.' Create the Trigger: Attach a `FOR EACH STATEMENT` trigger named `trg_audit_students_statement` to the `Students` table that executes the function you created in Requirement 1.
- Testing Instructions: After implementing the function and the trigger, write a single SQL `UPDATE` statement that affects multiple rows to verify your trigger works correctly.

Term project

6 Project description

6.1 Part I

- Provide a report on all the required tasks of your PostgreSQL labs. This later must include the ultimate ERD of the university database.
- All the SQL code should be well documented (with comments), including triggers, and provided in a separate file with the extension '.sql'.

6.2 Part II

The goal of this part of the project is as follows.

- Extend the University database schema to manage students assignments to groups and sections, their enrollments to courses, their attendance to the different activities and exams running (to handle students' marks and exams' schedule) and eventually manage activities' schedules.
- The ultimate goal is to develop a python application that provides the required features described below.

6.2.1 ERD design and Normalization

- Revise the university ERD, the ultimate ERD obtained when taking into consideration all the required work including section 2.4, as follows:
 1. Drop the entity department while reflecting this change on the ERD, and
 2. ADD the following attributes to the entity Instructor:
 - Department name
 - Department building
 - Department budget
 3. Provide the new ERD
 4. Map the proposed ERD to a relational schema.
 5. Is the obtained relational schema in 2NF, 3NF, BCNF? Discuss. If not so, provide a BCNF-decomposition and discuss whether the decomposition is lossless or not.
 6. Provide a report on the required work of this section.

6.2.2 Back to the ultimate database schema before section 6.2.1

1. Extend the ERD to include courses' activities. Each course must have a Lecture activity (mandatory), and may have a Tutorial work activity and/or a Practical work activity.
2. Extend the ERD to include students' marks management and exams running (room reservation, dates and time slots).
3. Map the ERD to a relational schema.
4. Use PostgreSQL (mandatory) to update the university database schema used in your labs.

6.2.3 Python application

1. Develop a python application with a graphic interface that offers the following features:
 - A sub-menu that handles all CRUD operations on all the tables (Student, Instructor, Course, Department, etc.). Note that, each CRUD operation should be handled through an input screen menu.
 - A sub-menu to manage assignment of modules to teaching staff and reservations.
 - A sub-menu to manage student marks, and their attendance to the different activities of the modules.
 - A sub-menu to manage students grading (results processing) while considering the failing grade for each module (the disqualifying mark for each module).
 - A sub-menu to display the results of following SQL queries:
 - (a) The list of students by group.
 - (b) The list of students by section.
 - (c) The time table of each instructor.
 - (d) The time table of students by section, then by group.
 - (e) The list of students who passed the semester.
 - (f) The list of disqualifying mark by module.
 - (g) The list of the average marks by course and by group
 - (h) The students who received a failing grade in a module.
 - (i) The students eligible for a resit in a module.
 - (j) List of students excluded from the module
 - A sub-menu "Audit" that allows to audit data manipulation operations (INSERT, UPDATE, and DELETE) on the student marks and student attendance using statement triggers. An audit table should be created and a single row must be inserted into the audit table on each required operation. This table must include the following attributes:
 - (a) **OperationType**: The type of SQL operation performed (e.g., 'INSERT', 'UPDATE', 'DELETE').
 - (b) **OperationTime**: The current timestamp (CURRENT_TIMESTAMP).
enumerate
2. **Important remarks**: The sql queries from (a) to (j) should include at least five functions.

6.3 Deliverables

1. You should submit the following deliverables:
 - A well-organized report including all the requested tasks.
 - For the SQL queries, functions and triggers you should submit a file with '.sql' extension.
 - The python code should be submitted in a dedicated folder.
2. Submission of the project
 - The delegate of each group should submit a zip file including each student's team work of his group.
 - The delegate of each section should provide the instructor in charge of the practical work of each group with the list of project partners (that do not exceed two students by team), as soon as possible.
 - The deliverables should be submitted to the instructor in charge by January 6 2026.

Don't forget to stay organized and work as a team.

7 ANNEXES

7.1 Some PostgreSQL data types

PostgreSQL provides a rich set of built-in data types to store various kinds of information. These can be broadly categorized as follows:

- Numeric Types:
 1. Integers: smallint, integer (or int), bigint for varying sizes of whole numbers.
 2. Auto-incrementing Integers: smallserial, serial, bigserial which automatically generate unique, sequential numbers.
 3. Floating-Point Numbers: real (single precision) and double precision (double precision) for approximate numeric values.
 4. Exact Numeric: numeric (or decimal) for numbers with selectable precision and scale, suitable for financial calculations.
 5. Monetary: money for currency amounts.
- Character Types:
 1. Fixed-length: char(n) (or character(n)) stores strings of a specified length, padded with spaces if shorter.
 2. Variable-length: varchar(n) (or character varying(n)) stores strings up to a specified maximum length.
 3. Unlimited Length: text stores strings of arbitrary length without a specified limit.
- Binary Data Types:
 1. Binary Strings: bytea stores binary data (e.g., images, audio) in hexadecimal format.
- Date/Time Types:
 1. Date: date stores calendar dates.
 2. Time: time stores time of day (without time zone).
 3. Time with Time Zone: timetz stores time of day with time zone information.
 4. Timestamp: timestamp stores date and time (without time zone).
 5. Timestamp with Time Zone: timestamptz stores date and time with time zone information.
 6. Interval: interval stores time spans.
- Boolean Type:
 1. boolean (or bool) stores truth values: TRUE, FALSE, or NULL.
- Structured Data Types:
 1. JSON: json stores JSON data as text.
 2. Binary JSON: jsonb stores JSON data in a more efficient binary format, offering better performance for querying.
 3. XML: xml stores XML data.
 4. Arrays: Allows storing variable-length multidimensional arrays of any built-in, user-defined, enum, or composite data type.
 5. hstore: Stores key-value pairs within a single column.
- Network Address Types: inet for IPv4 or IPv6 host addresses, cidr for IPv4 or IPv6 network addresses, and macaddr for MAC addresses.
- etc.

7.2 ANNEX I - Tables to create using psql tool

```

CREATE TABLE Department(
Department_id integer,
name varchar(25) NOT NULL,
CONSTRAINT UN_Department_Name UNIQUE (name),
CONSTRAINT PK_Department PRIMARY KEY(Department_Id)
);
CREATE TABLE Student(
Student_ID integer,
Last_Name varchar(25) NOT NULL,
First_Name varchar(25) NOT NULL,
DOB date NOT NULL,
Address varchar(50) DEFAULT NULL,
City varchar(25) DEFAULT NULL,
Zip_Code varchar(9) DEFAULT NULL,
Phone varchar(10) DEFAULT NULL,
Fax varchar(10) DEFAULT NULL,
Email varchar(100) DEFAULT NULL,
CONSTRAINT PK_Student PRIMARY KEY (Student_ID)
);
CREATE TABLE Course(
Course_ID int4 NOT NULL,
Department_ID int4 NOT NULL,
name varchar(60) NOT NULL,
Description varchar(1000),
CONSTRAINT PK_Course PRIMARY KEY (Course_ID, Department_ID),
CONSTRAINT "PK_Course_Department" FOREIGN KEY (Department_ID) REFERENCES Department (Department_ID)
ON UPDATE RESTRICT ON DELETE RESTRICT
);
CREATE TABLE Instructor(
Instructor_ID integer,
Department_ID integer NOT NULL,
Last_Name varchar(25) NOT NULL,
First_Name varchar(25) NOT NULL,
Rank varchar(25),
CONSTRAINT CK_Instructor_Rank CHECK (Rank IN ('Substitute','MCB', 'MCA', 'PROF')),
Phone varchar(10) DEFAULT NULL,
Fax varchar(10) DEFAULT NULL,
Email varchar(100) DEFAULT NULL,
CONSTRAINT PK_Instructor NOT NULL (Instructor_ID),
CONSTRAINT "FK_Instructor_Department_ID" FOREIGN KEY (Department_ID) REFERENCES Department (Department_ID)
ON UPDATE RESTRICT ON DELETE RESTRICT );
CREATE TABLE Room(
Building varchar(1),
RoomNo varchar(10),
Capacity integer CHECK (Capacity > 1),
CONSTRAINT PK_Room PRIMARY KEY (Building, RoomNo)
);
CREATE TABLE Reservation(
Reservation_ID integer, Building varchar(1) NOT NULL,
RoomNo varchar(10) NOT NULL,
Course_ID integer NOT NULL,

```

```

Department_ID integer NOT NULL,
Instructor_ID integer NOT NULL,
Reserv_Date date NOT NULL DEFAULT CURRENT_DATE,
Start_Time time NOT NULL DEFAULT CURRENT_TIME,
End_Time time NOT NULL DEFAULT '23:00:00',
Hours_Number integer NOT NULL,
CONSTRAINT PK_Reservation NOT NULL (Reservation_ID),
CONSTRAINT "FK_Reservation_Room" FOREIGN KEY (Building,RoomNo) REFERENCES Room
(Building,RoomNo)
ON UPDATE RESTRICT ON DELETE RESTRICT,
CONSTRAINT "FK_Reservation_Course" FOREIGN KEY (Course_ID, Department_ID) REFERENCES
Course (Course_ID,Department_ID)
ON UPDATE RESTRICT ON DELETE RESTRICT,
CONSTRAINT "FK_Reservation_Course" FOREIGN KEY(Instructor_ID) REFERENCES Instruc-
tor (Instructor_ID)
ON UPDATE RESTRICT ON DELETE RESTRICT,
CONSTRAINT CK_Reservation_Hours_Number CHECK (Hours_Number >= 1),
CONSTRAINT CK_Reservation_StartEndTime CHECK (Start_Time <End_Time)
);

```

7.3 Remarks

ON UPDATE RESTRICT ON DELETE RESTRICT postgresql: ON UPDATE RESTRICT and ON DELETE RESTRICT are actions applied to a foreign key constraint to prevent updates or deletions of a row in the parent table if there are existing dependent rows in the child table. This helps maintain referential integrity.

- **ON DELETE RESTRICT:** If you try to delete a row in the parent table (e.g., deleting a product from a products table), the operation will fail with an error if any rows in the child table still reference that parent row.
- **UPDATE RESTRICT:** If you try to update the primary key value of a row in the parent table, the operation will fail if any rows in the child table still reference that old key value.
- **Note: RESTRICT** is the default behavior for both ON DELETE and ON UPDATE actions if you do not explicitly specify any action when defining a foreign key constraint in PostgreSQL.

ON UPDATE CASCADE, ON DELETE CASCADE Actions applied to a foreign key constraint.

- **ON DELETE CASCADE** If you delete a row in the parent table, the database system will automatically delete all corresponding rows in the child table.
- **ON UPDATE CASCADE** If you modify the value of the primary key in the parent table, the database system will automatically update the corresponding foreign key value in all child rows.

7.4 ANNEX II - Tuples insertion

7.4.1 Try the following commands.

- Department:


```

INSERT INTO Department VALUES ('1','SADS');
INSERT INTO Department VALUES ('2','CCS');
INSERT INTO Department VALUES ('3','GRC');
INSERT INTO Department VALUES ('4','INS');

```

– Student:

INSERT INTO Student VALUES

('1','Ali', 'Ben Ali','1979/02/18','50, 1st street','Algiers','16000','0143567890',NULL,'A1@yahoo.fr');

INSERT INTO Student VALUES

('2','Amar', 'Ben Ammar','1980/08/23','10, Avenue b','BATNA','05000','0678567801',NULL,'pt@yahoo.fr');

INSERT INTO Student VALUES

('3','Ameur', 'Ben Ameur','1978/05/12','25, 2nd street','Oran','31000','0145678956','0145678956','o@yahoo.fr');

INSERT INTO Student VALUES

('4','Aissa', 'Ben Aissa','1979/07/15','56, Road','Annaba','23000','0678905645',NULL,'d@hotmail.com');

INSERT INTO Student VALUES

('5','Fatima', 'Ben Abdedallah','1979/08/15','45, Faubourg','Constantine','25000',NULL,NULL,NULL);

– Instructor:

INSERT INTO Instructor VALUES('1','1','Abbas','BenAbbes','MCA','4185','4091','Ab@yahoo.fr');

INSERT INTO Instructor VALUES('2','1','Mokhtar','BenMokhtar','Substitute', NULL, NULL, NULL);

INSERT INTO Instructor VALUES('3','1','Djemaa','Ben Mohamed','MCB', NULL, NULL, NULL);

INSERT INTO Instructor VALUES('4','1','Lahlou','Mohamed','PROF', NULL, NULL, NULL);

INSERT INTO Instructor VALUES('5','1','Abla','Chad','MCA',NULL,NULL,'ab@lgmail.com');

INSERT INTO Instructor VALUES('6','4','Mariam','BALI','Substitue',NULL,NULL,NULL);

– Room:

INSERT INTO Room VALUES('B','020','15');

INSERT INTO Room VALUES('B','022','15');

INSERT INTO Room VALUES('A','301','45');

INSERT INTO Room VALUES('C','Lecture Hall 1','500');

INSERT INTO Room VALUES('C','Lecture Hall 2','200');

– Course:

INSERT INTO Course VALUES

('1','1','Databases','Licence(L3) : Modeling E/A and UML, Relational Model, Relational Algebra, Relational calculs,SQL, NFs and FDs');

INSERT INTO Course VALUES ('2','1','C++ progr.','Level Master 1');

INSERT INTO Course VALUES

('3','1','Advanced DBs','Level Master 2 -Program Licence and Master 1');

INSERT INTO Course VALUES ('4','4','English','');

– Reservation:

INSERT INTO Reservation VALUES ('1','B','022','1','1','1','2006/10/15','08:30:00','11:45:00','3');

INSERT INTO Reservation VALUES ('2','B','022','1','1','4','2006/11/04','08:30:00','11:45:00','3');

INSERT INTO Reservation VALUES ('3','B','022','1','1','4','2006/11/07','08:30:00','11:45:00','3');

INSERT INTO Reservation VALUES ('4','B','020','1','1','5','2006/10/20','13:45:00','17:00:00','3');

INSERT INTO Reservation VALUES ('5','B','020','1','1','4','2006/12/09','13:45:00','17:00:00','3');

INSERT INTO Reservation VALUES ('6','A','301','2','1','1','2006/09/02','08:30:00','11:45:00','3');

INSERT INTO Reservation VALUES ('7','A','301','2','1','1','2006/09/03','08:30:00','11:45:00','3');

INSERT INTO Reservation VALUES ('8','A','301','2','1','1','2006/09/10','08:30:00','11:45:00','3');

INSERT INTO Reservation VALUES ('9','A','301','3','1','1','2006/09/24','13:45:00','17:00:00','3');

INSERT INTO Reservation VALUES ('10','B','022','3','1','1','2006/10/15','13:45:00','17:00:00','3');

INSERT INTO Reservation VALUES ('11','A','301','3','1','1','2006/10/01','13:45:00','17:00:00','3');

INSERT INTO Reservation VALUES ('12','A','301','3','1','1','2006/10/08','13:45:00','17:00:00','3');

INSERT INTO Reservation VALUES ('13','B','022','1','1','4','2006/11/03','13:45:00','17:00:00','3');

INSERT INTO Reservation VALUES ('14','B','022','1','1','5','2006/10/20','13:45:00','17:00:00','3');

```
INSERT INTO Reservation VALUES ('15','B','022','1','1','4','2006/12/09','13:45:00','17:00:00','3');
INSERT INTO Reservation VALUES ('16','B','022','1','1','4','2006/09/03','08:30:00','11:45:00','3');
INSERT INTO Reservation VALUES ('17','B','022','1','1','5','2006/09/10','08:30:00','11:45:00','3');
INSERT INTO Reservation VALUES ('18','B','022','1','1','4','2006/09/24','13:45:00','17:00:00','3');
INSERT INTO Reservation VALUES ('19','B','022','1','1','5','2006/10/01','13:45:00','17:00:00','3');
INSERT INTO Reservation VALUES ('20','B','022','1','1','1','2006/10/08','13:45:00','17:00:00','3');
INSERT INTO Reservation VALUES ('21','B','022','1','1','4','2003/09/02','08:30:00','11:45:00','3');
```

7.4.2 Remarks

- **Insert multiple rows in one command** : For performance, it is more efficient to insert multiple rows in a single statement by providing comma-separated value lists.
- Example: Sql code

```
INSERT INTO products (name, price, last_update)
VALUES
('Monitor', 450.00, '2025-12-01'),
('USB Cable', 15.50, '2025-11-28'),
('Hard Drive', 90.99, '2025-12-02');
```